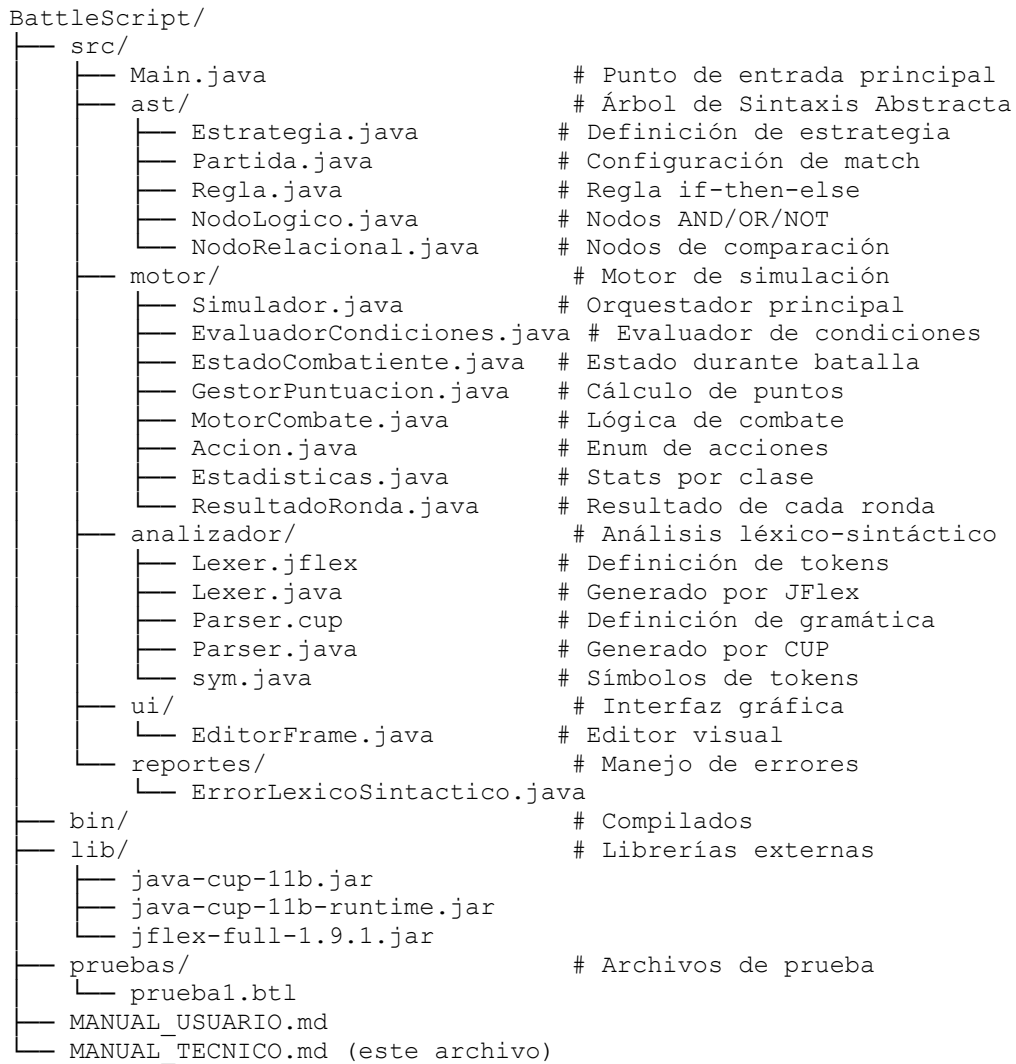


Manual Técnico - BattleScript

Arquitectura del Proyecto



Componentes Principales

1. AST (Árbol de Sintaxis Abstracta)

Estrategia (ast/Estrategia.java)

- Almacena nombre, clase, acciones iniciales, reglas
- Estructura jerárquica: nombre → clase → reglas

Regla (ast/Regla.java)

- Representa `if condición then acción`
- La condición es un Object (puede ser `NodoLogico` o `NodoRelacional`)
- La acción es un String (nombre de la acción)

NodoLogico (`ast/NodoLogico.java`)

- Representa operaciones lógicas: AND, OR, NOT
- Árbol binario (izquierda y derecha)

NodoRelacional (`ast/NodoRelacional.java`)

- Representa comparaciones: `==`, `!=`, `>`, `<`, `>=`, `<=`
- Compara dos expresiones

2. Motor de Simulación

Simulador (`motor/Simulador.java`)

- Orquestador principal de la batalla
- Ejecuta rondas secuencialmente
- Ronda 0: Acciones iniciales (sin evaluación de reglas)
- Rondas 1-N: Evaluación de reglas + resolución de acciones

EvaluatorCondiciones (`motor/EvaluadorCondiciones.java`)

- Evalúa recursivamente el árbol de condiciones
- Resuelve variables de estado
- Maneja operadores lógicos y relacionales

EstadoCombatiente (`motor/EstadoCombatiente.java`)

- Almacena estado actual: vida, recurso, puntos, historial
- Mantiene bonificaciones temporales (defensa, WAR_CRY)
- Limita vida y recurso a máximos

MotorCombate (`motor/MotorCombate.java`)

- Calcula daño (físico/mágico)
- Aplica reducciones por defensa
- Determina prioridad de acciones
- Aplica curaciones y recuperaciones

GestorPuntuacion (`motor/GestorPuntuacion.java`)

- Calcula puntos por cada tipo de acción
- Detecta combos
- Aplica bonos de victoria
- Implementa penalizaciones

Accion (motor/Accion.java)

- Enum con 10 acciones
- Propiedades: nombre, poder, costo, tipo, prioridad

3. Análisis Léxico-Sintáctico

Lexer.jflex (analizador/Lexer.jflex)

- Define tokens: palabras clave, operadores, literales
- Procesado por JFlex → `Lexer.java`

Parser.cup (analizador/Parser.cup)

- Define gramática BNF
- Especifica producciones de AST
- Procesado por CUP → `Parser.java` + `sym.java`

Flujo de Ejecución

Main.java

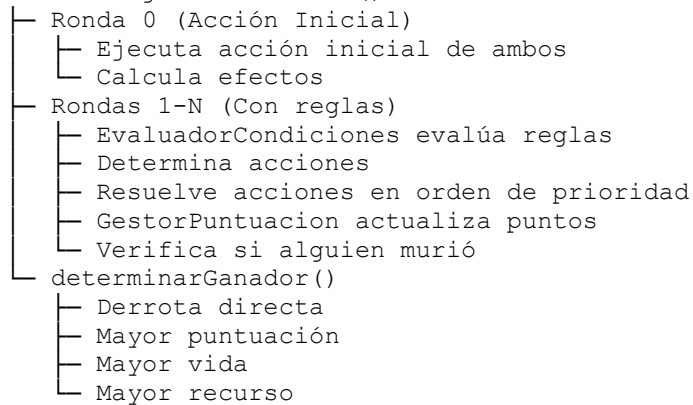
↓

Crea Estrategias (mago/guerrero)

Crea Partida (configuración)

↓

Simulador.ejecutarPartida()



↓

Retorna log formateado con resultado

Clases de Combatientes

Mago (mage)

- **Vida Máxima:** 103 HP
- **Recurso Máximo:** 110
- **Clase:** Especializado en magia
- **Bonus de Daño Mágico:** +15%

Guerrero (warrior)

- **Vida Máxima:** 137 HP
- **Recurso Máximo:** 90
- **Clase:** Especializado en física
- **Bonus de Defensa:** -15% daño recibido

Cálculo de Daño

Daño Físico

```
danoBase = poder + (poder * 0.1) * claseAtacante
danoConBonus = danoBase + bonificacionWARCRY
danoFinal = danoConBonus - (danoConBonus * 0.15 * claseDefensor)
```

Si defendiendo:
danoFinal *= 0.6 (reducción 40%)

Daño Mágico

```
danoBase = poder + (poder * 0.15) * claseAtacante
danoFinal = danoBase
```

Si defendiendo:
danoFinal *= 0.5 (reducción 50%)

Sistema de Puntuación

Cada acción genera puntos:

Acción	Fórmula	Puntos
Daño	damagePoint * dañoCausado	Variable
Curación	healingPoint * vidaRecuperada	Variable
Defensa	successfulDefense	Fijo
Combo	comboPoints	Fijo
Acción Fallida	-failedActionPenalty	Fijo
Victoria	victoryBonus + bonusVidaBaja	Variable

Generación de Código

JFlex (Lexer)

```
java -jar lib/jflex-full-1.9.1.jar -d src/analizador
src/analizador/Lexer.jflex
```

CUP (Parser)

```
java -cp lib/java-cup-11b.jar java_cup.Main -destdir src/analizador
src/analizador/Parser.cup
```

Compilación

```
javac -d bin -cp "lib/java-cup-11b-runtime.jar" $(find src -name "*.java")
```

Ejecución

```
java -cp "bin;lib/java-cup-11b-runtime.jar" Main
```

Estructura de Archivos de Prueba

```
mage NOMBRE {
    initial: ACCIÓN
    rules: [
        if (CONDICIÓN) then ACCIÓN,
        else: ACCIÓN
    ]
}

warrior NOMBRE {
    initial: ACCIÓN
    rules: [
        if (CONDICIÓN) then ACCIÓN,
        else: ACCIÓN
    ]
}

match NOMBRE {
    players: [ESTRATEGIA1, ESTRATEGIA2]
    rounds: NUM
    scoring: { ... }
    bonuses: { ... }
}

main {
    run [PARTIDA] with {
        seed: NÚMERO
    }
}
```

Notas Importantes

1. **Determinismo:** Con la misma seed, el resultado es idéntico
2. **Generadores Independientes:** Cada estrategia tiene su propio Random
3. **Historial:** Se registran acciones ejecutadas exitosamente
4. **Penalizaciones:** Acciones fallidas restan puntos pero no se registran en historial
5. **Ronda 0 Especial:** Las acciones iniciales se ejecutan sin evaluar reglas

Desarrollado para: Organización de Lenguajes y Compiladores 1

Año: 2026